tower_detector -- git-history "patch-tower" (S . . heuristic-tower) detector

Companion doc (S11.4.18 script-documentation mandate) for:

- `scripts/fastcycle/review/tower_detector.sh` (sh dispatcher)
- `scripts/fastcycle/review/lib/tower_detector_run.py` (the real implementation -- its own module docstring is the authoritative design reference; this doc is a reader-facing summary + the explicit false-positive/false-negative boundary + the recommended human/agent response to a flag)
- `scripts/fastcycle/tests/test_tower_detector_red.sh` (TDD-first RED/GREEN suite, unit + golden-good/golden-bad/negative-control integration cases)

What it detects

This project's own research finding (`docs/research/fast_dev_cycles_acceleration_2026-10/FINDINGS.md` , S6, shortlist item 4) names the mechanism directly:

> A git-history tower detector flags any symbol with 3+ branch-adding commits under one item in a cycle -- mechanically triggers S11.4.250 before the 3rd patch round.

Three tracked SpecKit-004 stories in this very repository's history demonstrate the failure mode this tool exists to catch early:

Item	File	Real diagnosis round	Compensating layers before diagnosis
T177	`scripts/fastcycle/consumers/migrate.sh`	round 19-20 (data-integrity hardening commit `7322d08`)	~8 rounds of successive narrow patches
T048	`scripts/fastcycle/tests/test_fc_timer_golden_output.sh`	round 20-21 (architectural rewrite, commit `b348b40`)	7 enumerated layers (noise-floor classification -> known-flaky registry -> exit-code noise floor -> summary-tail truncation -> registry exact-accounting -> nonzero-delta clause -> unconditional ordering + digit bound), file grew 461 -> 1352 lines
T085	several shared primitives	round 3-4 (R4-B1 BLOCKING finding)	fewer rounds -- this item moved fast already

In every one of the first two cases, an independent reviewer eventually diagnosed a **S11.4.250 "heuristic-tower" pattern** -- many successive, narrow, special-case-adding commits compensating for ONE unfixed root defect, instead of a genuine architectural fix -- and told the fixer to stop patching and rebuild properly. The diagnosis came late because nothing *mechanically* watched for the pattern; it took a human reading the whole file's history to notice the shape. `tower_detector` makes that observation automatic and early: it walks a tracked item's own commit history and flags the exact moment a **3rd qualifying branch-adding commit** lands

on the same symbol, so the S11.4.250 response (stop, invoke `superpowers:systematic-debugging` / S11.4.102 root-cause investigation on the real underlying defect) can happen *before* review round N+1 has to re-discover it by hand.

How it decides "item" and "symbol" (real, not invented)

- **Item grouping**: this repository's own commit-message convention (verified against real `git log` output before writing this tool, not assumed) names a SpecKit task id (`T048` , `T085` , `T177` , ...) or, in a main-repo checkout, an S11.4.54 `ATM-NNN` id, as a literal token somewhere in the subject line -- sometimes as a Conventional-Commits scope suffix (`T085-r5`), sometimes as free text (`T048): round 11`). `tower_detector` matches the requested item id as a plain **word-boundary token** anywhere in the subject, which is robust to both real forms this repo's own history actually uses.
- **Symbol boundary**: `git diff -U0` 's own per-hunk function-context line (git's built-in "which function is this hunk inside" heuristic, already proven on this very codebase -- a real `git diff -U0` run against `migrate.sh` correctly reported `write_out() {` / `not_migrated() {` as hunk context) is normalized to `<file>::<symbol>` **only when that context line itself matches an explicit function/ class/def DEFINITION pattern** (shell `name() {` , python `def name(` / `class Name` , go `func name(` , rust `fn name(` , javascript/ typescript `function name(`). Every other context -- empty, a bare control-flow keyword like `fi` / `if` / `for` / `while` / `done` , OR (round 1 independent review finding R1-review-I2, measured on this repo's own real T177/ T048/T085 history) an arbitrary non-keyword token git's heuristic happened to surface nearby (`echo` , `import` , `PYEOF` , `trap` , and others were all silently accepted as fake symbol names by an earlier revision's generic fallback) -- coarsens to the explicitly-sanctioned **`FILE:<path>`** fallback. There is deliberately no "accept the first identifier-shaped token" fallback any more: a context line not matching a real definition boundary carries no attribution value, and inventing a fake per-line symbol from it was worse than honestly coarsening.

How a commit qualifies as "branch-adding" for a symbol

Both conditions below must hold (AND-NOT structure, by design -- neither condition alone is a reliable signal):

1. **Diff shape**: the net count of added-minus-removed lines opening a conditional branch (`if` / `elif` / `else if` / `case ... in` / `except` / `catch`) in that symbol's hunk(s) is `>= 1` , **and** the hunk does not delete roughly as much as it adds (default: deleted lines `< 0.8 x` added lines) -- a hunk that deletes almost as much as it adds is the "remove the old special case, add a different one" *replace* shape, not the "layer another special case on top" shape.
2. **Message shape**: the commit's own Conventional-Commits type (optionally carrying a trailing `!` breaking-change marker, e.g. `refactor!:` or `refactor(x)!:` -- matched and discarded, round 2 independent review NEW-4 doc-sync fix) is not `refactor` / `revert` , and its description does not start with `remove` / `replace` / `delete` / `revert` / `refactor` / `drop` / `simplify` / `rewrite` / `consolidate` -- a commit whose *stated intent* is removal/rewrite is excluded even when its diff superficially matches, since S11.4.250's whole point is distinguishing "fixed the primitive and removed the cascade" from "added another layer".

A symbol accumulating `>= 3` (default; `--min-branch-commits`) such qualifying commits under the requested item/range/path is **flagged**.

Usage

```
# By tracked workable-item id (word-boundary token match on commit subjects):
scripts/fastcycle/review/tower_detector.sh --item T085 --out report.json

# Narrowed to one file/symbol-space:
scripts/fastcycle/review/tower_detector.sh --item T177 \
    --path scripts/fastcycle/consumers/migrate.sh --out report.json

# By an explicit commit range instead of (or together with) an item:
scripts/fastcycle/review/tower_detector.sh --range v1.0..HEAD --out report.json

# Tunable threshold / refactor-ratio:
scripts/fastcycle/review/tower_detector.sh --item T048 \
    --min-branch-commits 2 --refactor-ratio 0.6 --out report.json
```

Exit codes: `0` zero findings; `1` >= 1 finding (**advisory**, see below); `2` usage error (no `--item` / `--range` / `--path` given, or a bad `--min-branch-commits` / `--refactor-ratio` value); `3` a genuine git command failure (bad `--range`, not a repository); `4` (dispatcher-only) no `python3` / `python` interpreter found on `PATH`.

ADVISORY ONLY -- never a gate (S . .)

This tool's non-zero exit on a finding is informational, matching this project's own house convention for precheck-style tools (e.g. `review/precheck_pack.sh`: "1 any check FAILs"). **It is not wired into any build, review, or dispatch gate.** Per S11.4.269, a mechanical signal like this one must never auto-gate or block on its own judgment -- it can be wrong in both directions (see "Honest limitations" below), so it *informs*, it never *decides*. If a future change wires this tool into a gate sequence or a review-dispatch guard, that wiring MUST remain advisory-only (surface the finding, never refuse a build/commit/dispatch on this tool's verdict alone).

Honest limitations (false-positive / false-negative boundary, S . .)

False negatives (will miss):

- A bare early-return/guard-clause addition with *no* branch keyword on the same added line (e.g. a lone `return 1` whose guarding `if` was added in an *earlier* commit) is not detected -- only the enumerated branch-opener keyword set is matched, by design (the primary signal the research brief named), not arbitrary control-flow changes.
- A tower spread across *symbols* git's own context heuristic attributes inconsistently (nested blocks sometimes misattributed to the wrong enclosing function) may split what is really one tower's commit count across two reported symbols, delaying the threshold.
- Item-token matching reads commit **subjects** only, not bodies -- a project whose convention puts the item id only in the commit body would see every one of its commits read as item-less by `--item` (never crashing; simply excluded, same as "zero matching commits").
- A branch opened with a guard combinator (`&&` / `||`) rather than a bare `if` / `elif` / `case` / `except` / `catch` keyword, or control flow expressed via `while` / `switch` / `when` (deliberately scoped OUT by the design brief's own enumerated keyword list, not merely overlooked) is never counted as branch-adding.
- `--no-merges` means a tower built entirely inside a sequence of merge-conflict RESOLUTIONS (the compensating logic lands only in merge commits, never a plain commit) is invisible to this tool.
- `normalize_symbol()` 's DEFINITION patterns require a same-line opening brace/paren shape (e.g. shell `name() {` on one line); a next-line- brace style (`name()` then `{` on the following line), `async def` , `pub async fn` , `pub(crate) fn` , a Go generic (`func F[T any](`), or `export default function` all safely coarsen to `FILE:` rather than being attributed to their real symbol (round 2 independent review, IMPORTANT-2 follow-up measurement) -- a false negative at the symbol- granularity layer, never a fake symbol; the FILE-level aggregate still sees the hunk.

- The `diff --git a/(.+) b/(.+)` path-extraction regex in `parse_diff_hunks()` could mis-split a path containing the literal substring `" b/"` -- an edge case not currently guarded against.

False positives (may over-flag):

- git's own function-context heuristic is itself a best-effort regex guess for most languages; an occasional hunk can be mis-attributed to an unrelated nearby symbol, inflating that symbol's count. **Round 1 independent review fix (R1-review-I2)**: an earlier revision also accepted *any* identifier-shaped token in a non-definition context line as a fake symbol name -- measured against this repo's own real history, the most common such "symbol" was literally `echo` (56 hunks), plus `import`, `PYEOF`, `trap`, `git`, `cat`, `rm`, `cp`, `EOF`, `printf`, `sys.exit`, `HERE`. Fixed: `normalize_symbol()` now accepts ONLY lines matching an explicit function/class/def DEFINITION pattern (shell, python, go, rust, javascript/typescript); every other context coarsens to the `FILE:` fallback instead of inventing a fake per-line symbol. This is a *reduction* in false-positive symbol noise, not a complete elimination of mis-attribution -- git's heuristic can still occasionally attribute a hunk to the wrong *real* definition line when hunks are closely nested.
- Two *independently legitimate* features landing 3+ `if`-adding commits on the same symbol under one item (not actually compensating for the same root defect, just three separate well-reasoned features) will still be flagged -- the tool has no way to distinguish "three compensating patches for one bug" from "three unrelated, well-designed features that happen to touch the same function". **This is why the tool's output is a recommendation to *investigate*, never a verdict that the code is wrong.**
- **Prose counts as a branch (MINOR-4)**: `BRANCH_TOKEN_RE` matches line CONTENT only, with no awareness of code vs. comment vs. docstring vs. markdown prose -- a sentence beginning "If the caller passes..." or "Except where noted..." matches the same way a real `if` / `except` statement does. This tool is calibrated against code files (shell/ python/go/rust/js); running it against a predominantly-prose file (a `.md` doc, a changelog) can over-count. No per-file-type scoping is applied -- a known, documented gap, not a silent blind spot (this bullet is the doc-side half of that documentation; see `count_branch_tokens()`'s own docstring for the code-side half).

- **Regression-test accretion (named by round 1 independent review, MINOR-1)**: a dedicated per-round regression-test file (this project's own convention: `test_X_rN_regression.sh`) that legitimately grows a new test case each round looks, from git-history shape alone, exactly like a tower -- each round's commit genuinely adds a new conditional branch (a new test case), under the same item, to the same file. This is NOT a defect pattern (the file's whole *purpose* is to accrete), but `tower_detector` cannot currently distinguish it from a real tower; see the retrospective table below, which reports every such flag found on this session's own real history honestly rather than omitting the inconvenient ones.

Recommended response to a flag (S . . / S . .)

1. Read the finding's `qualifying_commits` and `all_touches` evidence trail -- do not act on the count alone.
2. If the pattern IS a real tower (each commit really is "another layer compensating for the same unfixed thing"): **stop** before writing a 4th patch. Invoke `superpowers:systematic-debugging` (S11.4.102) on the symbol's real underlying defect. Per commit `b348b40` 's own demonstrated remedy on `test_fc_timer_golden_output.sh` : fix the primitive at its source, then *delete* the whole compensating cascade and replace it with the simple rule that was actually needed -- S11.4.124 applies to the removal (git-history evidence citing the fix, never a silent delete).
3. If the pattern is NOT a real tower (three legitimate, independent changes that happen to share a symbol): record that judgment (e.g. in the commit message of the next change to that symbol, or in a review note) so a human/agent re-running this tool later understands why the flag was knowingly accepted -- this tool does not provide a suppression mechanism of its own (no per-symbol allowlist), by design, since S11.4.269 forbids treating its signal as authoritative in either direction.

Retrospective validation against this session's own real towers

Run read-only against this repository's own already-landed history (never rewriting it). Re-run and table updated after the round 1 independent review's R1-review-I2 symbol-attribution fix (below) -- the numbers here are the POST-FIX ones, reported honestly including every flag the tool genuinely produces, not a curated subset:

This table is **UNRESTRICTED** (`--item <ID>` only, no `--path` filter) for ALL THREE items (round 2 independent review NEW-1 fix: an earlier revision reported T177 through a `--path`-restricted query showing only 1 of its 5 real findings, while T048/T085 were already shown unrestricted -- an inconsistency corrected here by running every item the same way and reporting every finding each produces):

Item	Flagged symbol	Qual-ify-ing com-mits	Fired at round	Re hu ma di gr is ro
T177	`FILE:scripts/fastcycle/consumers/migrate.sh`	8	**round 3** (commit `05b6239`)	ro 19

Item	Flagged symbol	Qual-ify-ing com-mits	Fired at round	Re hu ma di gr is ro
T177	`scripts/fastcycle/consumers/audit.py::cmd_summary`	6	round 3 (commit `05b6239`)	n/a

Item	Flagged symbol	Qual-ify-ing com-mits	Fired at round	Re hu ma di gn is ro
T177	`FILE:...test_consumer_audit_red.sh`	7	round 3 (commit `05b6239`)	n/a
T177	`FILE:...test_consumer_enumerate_red.sh`	3	round 3 (commit `05b6239`)	n/a
T177	`FILE:...test_consumer_migrate_red.sh`	12	round 3 (commit `05b6239`)	n/a
T048	`FILE:scripts/fastcycle/tests/test_fc_timer_golden_output.sh`	9	**round 7** (commit `c512992`)	ro 20

Item	Flagged symbol	Qual-ify-ing com-mits	Fired at round	Re hu ma di gr is ro
T048	`...test_fc_timer_golden_output.sh::validate_triplet`	3	round 11 (commit `e2c92dd`)	ro 20
T048	`FILE:...test_metatest_per_mutant_r4_regression.sh`	5	round 7 (commit `c512992`)	n/a
T048	`FILE:...test_fc_timer_golden_output_r4_regression.sh`	3	round 13 (commit `df091b6`)	n/a

Item	Flagged symbol	Qual-ify-ing com-mits	Fired at round	Re hu ma di gr is ro
T048	`FILE:...tests/lib/golden_triplet_fixture.sh`	3	round 19 (commit `c48356b`)	n/a
T048	`FILE:...test_fc_timer_golden_output_r7_regression.sh`	3	round 19 (commit `c48356b`)	n/a

Item	Flagged symbol	Qual-ify-ing com-mits	Fired at round	Re hu ma di gr is ro
T085	`scripts/fastcycle/gates/catch-set_compare.py::compute_comparison`	3	round 3 (commit `b50e685`)	ro 3- (R B1
T085	`FILE:...test_catch-set_compare_seed_independence_r2_regression.sh`	3	round 5 (commit `10b7a06`)	n/a

Item	Flagged symbol	Qual-ify-ing com-mits	Fired at round	Re hu ma di gr is ro
T085	`FILE:...test_re-view_record_backfill_evidence_r2_regression.sh`	3	round 5 (commit `10b7a06`)	n/a

14 real findings total across the three items (5 for T177, 6 for T048, 3 for T085) -- stated as an exact count here specifically because an earlier revision of this doc, and this session's own CONTINUATION.md addendum, both mis-stated it as "10" (round 2 independent review NEW-1, second half: a genuine arithmetic/ transcription error, corrected here rather than silently left for a reader to re-derive).

Honest reading (S11.4.6): T085's genuine-defect result is reported faithfully as "roughly concurrent", not inflated to match the larger T177/T048 numbers. The regression/test-support-file flags under all three items are the regression-test-accretion false-positive class named above -- included here for honesty rather than silently dropped because they don't fit the "early detection" success story; a human/ agent reviewing a real flag on a dedicated test-support file should recognise this class and not treat it as a tower without checking. `audit.py:: cmd_summary` is a genuine, real finding this tool produces on real history, but it was never the subject of this session's own T177 tower diagnosis (that diagnosis centred on `migrate.sh`), so it has no human baseline to compare against and is reported here without a "rounds earlier" claim, never a fabricated one. The commands used to produce this table are reproducible read-only `git-log` / `git-show` invocations against this repository's own history; no file was modified to produce them.